

LAB EXPERIMENTS: TOPIC 4

TOPIC 4: DYNAMIC PROGRAMMING

1. Dice Throw Problem Using Dynamic Programming

Aim

To find the number of ways to get a target sum using a given number of dice.

Algorithm

1. Initialize DP table with `dp[0][0] = 1`.
2. Add every possible face value for each die.
3. Store the number of ways for each sum.
4. Display the required number of ways.

Pseudocode

```
dp[0][0] = 1
for each die:
    for each sum:
        for face = 1 to sides:
            dp[die][sum] += dp[die-1][sum-face]
return dp[dice][target]
```

Time Complexity

$O(D \times S \times T)$

Space Complexity

$O(D \times T)$

Code

```
def dice_ways(sides, dice, target):
    dp = [[0] * (target + 1) for _ in range(dice + 1)]
```

```
dp[0][0] = 1

for d in range(1, dice + 1):
    for s in range(1, target + 1):
        for face in range(1, sides + 1):
            if s >= face:
                dp[d][s] += dp[d-1][s-face]

return dp[dice][target]

print(dice_ways(6, 2, 7))
print(dice_ways(4, 3, 10))
```

Input

```
6 sides, 2 dice, target = 7
4 sides, 3 dice, target = 10
```

Output

```
6
27
```

Result

The number of ways to obtain the target sums was found successfully.

2. Assembly Line Scheduling

Aim

To find the minimum time required to process a product through two assembly lines.

Algorithm

1. Store station and transfer times.
2. Calculate the minimum time at every station for both lines.
3. Add entry and exit times.

4. Display the minimum total time.

Pseudocode

$f1[0] = e1 + a1[0]$

$f2[0] = e2 + a2[0]$

for $i = 1$ to $n-1$:

$f1[i] = a1[i] + \min(f1[i-1], f2[i-1] + t2[i-1])$

$f2[i] = a2[i] + \min(f2[i-1], f1[i-1] + t1[i-1])$

$answer = \min(f1[n-1] + x1, f2[n-1] + x2)$

Time Complexity

$O(n)$

Space Complexity

$O(n)$

Code

```
def assembly(a1, a2, t1, t2, e1, e2, x1, x2):
```

```
    n = len(a1)
```

```
    f1 = [0] * n
```

```
    f2 = [0] * n
```

```
    f1[0] = e1 + a1[0]
```

```
    f2[0] = e2 + a2[0]
```

```
    for i in range(1, n):
```

```
        f1[i] = a1[i] + min(f1[i-1], f2[i-1] + t2[i-1])
```

```
        f2[i] = a2[i] + min(f2[i-1], f1[i-1] + t1[i-1])
```

```
    return min(f1[-1] + x1, f2[-1] + x2)
```

```
print(assembly(
```

```
    [4,5,3,2,7,8],
```

```
    [2,10,1,4,5,3],
```

```
    [7,4,5,6,4],
```

```
    [9,2,3,1,2],
```

10,12,18,7
)

Input

Assembly line station, transfer, entry and exit times.

Output

35

Result

The minimum assembly time was found as **35 units**.

3. Assembly Line Scheduling with 3 Lines

Aim

To find the minimum production time using three assembly lines.

Algorithm

1. Store station times and transfer costs.
2. Calculate the minimum cost for each station and line.
3. Select the minimum previous cost plus transfer cost.
4. Display the minimum production time.

Pseudocode

```
dp[0][line] = station_time[0][line]
```

```
for each station:
```

```
    for each line:
```

```
        dp[station][line] =
```

```
        station_time + minimum(previous_cost + transfer_cost)
```

```
return minimum final cost
```

Time Complexity

$O(n \times L^2)$

Space Complexity

$O(n \times L)$

Code

```
station = [  
    [5, 6, 7],  
    [9, 8, 6],  
    [3, 4, 5]  
]  
  
transfer = [  
    [0,2,3],  
    [2,0,4],  
    [3,4,0]  
]  
  
dp = [5, 6, 7]  
  
for i in range(1, 3):  
    new = []  
    for j in range(3):  
        cost = min(dp[k] + transfer[k][j] for k in range(3))  
        new.append(cost + station[i][j])  
    dp = new  
  
print(min(dp))
```

Input

Station times and transfer matrix for 3 lines.

Output

17

Result

The minimum production time was found as **17 units**.

4. Minimum Path Distance Using Matrix Form

Aim

To find the minimum travelling distance using Dynamic Programming.

Algorithm

1. Represent distances using a matrix.
2. Use DP with visited cities.
3. Select the minimum unvisited city.
4. Return to the starting city.
5. Display the minimum distance.

Pseudocode

```
TSP(city, visited):  
    if all cities visited:  
        return distance[city][0]  
  
    minimum = infinity  
    for each unvisited city:  
        minimum = min(minimum,  
                        distance[city][next] + TSP(next, visited))  
    return minimum
```

Time Complexity

$O(n^2 \times 2^n)$

Space Complexity

$O(n \times 2^n)$

Code

```

#include <stdio.h>
#define INF 9999

int n = 4;
int dp[16][4];

int dist[4][4] = {
    {0,10,15,20},
    {10,0,25,25},
    {15,25,0,30},
    {20,25,30,0}
};

int tsp(int pos, int mask) {
    if (mask == (1 << n) - 1)
        return dist[pos][0];

    if (dp[mask][pos] != -1)
        return dp[mask][pos];

    int ans = INF;

    for (int city = 0; city < n; city++)
        if (!(mask & (1 << city)))
            ans = ans < dist[pos][city] +
                tsp(city, mask | (1 << city))
                ? ans : dist[pos][city] +
                tsp(city, mask | (1 << city));

    return dp[mask][pos] = ans;
}

int main() {
    for(int i=0;i<16;i++)
        for(int j=0;j<4;j++)
            dp[i][j] = -1;

    printf("Minimum Distance = %d", tsp(0,1));
    return 0;
}

```

Input

A 4×4 distance matrix.

Output

Minimum Distance = 80

Result

The minimum travelling distance was calculated successfully.

5. Travelling Salesman Problem with 5 Cities

Aim

To find the shortest route visiting all five cities exactly once.

Algorithm

1. Represent cities using a distance matrix.
2. Use DP with a visited-city mask.
3. Find the minimum route.
4. Return to the starting city.

Pseudocode

```
TSP(city, visited):  
    if all cities visited:  
        return distance[city][start]  
  
    minimum = infinity  
    for every unvisited city:  
        calculate route cost  
    return minimum
```

Time Complexity

$O(n^2 \times 2^n)$

Space Complexity

$O(n \times 2^n)$

Code

```
from functools import lru_cache
```

```
D = [  
    [0,10,15,20,25],  
    [10,0,35,25,30],  
    [15,35,0,30,20],  
    [20,25,30,0,15],  
    [25,30,20,15,0]  
]
```

```
@lru_cache(None)  
def tsp(city, mask):  
    if mask == (1 << 5) - 1:  
        return D[city][0]  
  
    return min(  
        D[city][nxt] + tsp(nxt, mask | (1 << nxt))  
        for nxt in range(5) if not (mask & (1 << nxt))  
    )  
  
print(tsp(0, 1))
```

Input

Five-city distance matrix.

Output

85

Result

The shortest travelling route has a total distance of **85 units**.

6. Longest Palindromic Substring

Aim

To find the longest palindrome present in a given string using Dynamic Programming.

Algorithm

1. Create a DP table.
2. Mark single characters as palindromes.
3. Check substrings of increasing length.
4. Store the longest palindrome.

Pseudocode

for every character:

 mark as palindrome

for length = 2 to n:

 check first and last characters

 if equal and inside part is palindrome:

 mark substring as palindrome

return longest substring

Time Complexity

$O(n^2)$

Space Complexity

$O(n^2)$

Code

```
def longest_palindrome(s):  
    n = len(s)  
    dp = [[False]*n for _ in range(n)]  
    start, maxlen = 0, 1
```

```

for i in range(n):
    dp[i][i] = True

for length in range(2, n+1):
    for i in range(n-length+1):
        j = i + length - 1

        if s[i] == s[j] and (length == 2 or dp[i+1][j-1]):
            dp[i][j] = True
            if length > maxlen:
                start, maxlen = i, length

return s[start:start+maxlen]

print(longest_palindrome("babad"))
print(longest_palindrome("cbbd"))

```

Input

babad
cbbd

Output

bab
bb

Result

The longest palindromic substring was found successfully.

TOPIC 4: DYNAMIC PROGRAMMING — CONTINUATION

7. Longest Substring Without Repeating Characters

Aim

To find the length of the longest substring without repeated characters.

Algorithm

1. Use a set to store characters.
2. Move through the string using two pointers.
3. Remove repeated characters.
4. Track the maximum length.

Pseudocode

left = 0

for right in string:

 while character repeats:

 remove string[left]

 left++

 add character

 update maximum length

Time Complexity

$O(n)$

Space Complexity

$O(n)$

Code

```
def longest(s):
```

```
    seen = set()
```

```
    left = ans = 0
```

```
for right in range(len(s)):
    while s[right] in seen:
        seen.remove(s[left])
        left += 1
    seen.add(s[right])
    ans = max(ans, right-left+1)

return ans
```

```
print(longest("abcabcbb"))
print(longest("bbbbbb"))
print(longest("pwwkew"))
```

Input

abcabcbb, bbbbb, pwwkew

Output

3

1

3

Result

The longest non-repeating substring length was found successfully.

8. Word Break Problem

Aim

To check whether a string can be formed using words from a dictionary.

Algorithm

1. Create a DP array.
2. Set `dp[0] = True`.
3. Check every dictionary word.
4. Mark the position as possible if the word matches.

Pseudocode

```
dp[0] = true
```

```
for i = 1 to n:
```

```
    for every word:
```

```
        if word ends at i and dp[start] is true:
```

```
            dp[i] = true
```

```
return dp[n]
```

Time Complexity

$O(n^2)$

Space Complexity

$O(n)$

Code

```
def word_break(s, words):
```

```

dp = [False] * (len(s)+1)
dp[0] = True

for i in range(1, len(s)+1):
    for w in words:
        if i >= len(w) and dp[i-len(w)] and s[i-len(w):i] == w:
            dp[i] = True
return dp[-1]

print(word_break("leetcode", ["leet", "code"]))
print(word_break("applepenapple", ["apple", "pen"]))
print(word_break("catsanddog", ["cats", "dog", "sand", "and", "cat"]))

```

Input

Dictionary words and input string.

Output

True

True

False

Result

The string segmentation was checked successfully.

9. String Segmentation Using Dictionary

Aim

To determine whether a string can be segmented using the given dictionary.

Algorithm

1. Store the dictionary words.
2. Use DP to check valid prefixes.
3. Continue until the complete string is checked.
4. Display Yes or No.

Pseudocode

```
dp[0] = true
for i = 1 to n:
    for j = 0 to i:
        if dp[j] and s[j:i] is in dictionary:
            dp[i] = true
return dp[n]
```

Time Complexity

$O(n^2)$

Space Complexity

$O(n)$

Code

```
def segment(s, dictionary):
    dp = [False] * (len(s)+1)
    dp[0] = True
```



```
for i in range(1, len(s)+1):  
    for j in range(i):  
        if dp[j] and s[j:i] in dictionary:  
            dp[i] = True  
return dp[-1]
```

```
words = {"i","like","sam","sung","samsung","mobile",  
        "ice","cream","icecream","man","go","mango"}
```

```
print("Yes" if segment("ilike", words) else "No")  
print("Yes" if segment("ilikesamsung", words) else "No")
```

Input

```
ilike, ilikesamsung
```

Output

Yes

Yes

Result

Both strings can be segmented using the given dictionary.

10. Text Justification

Aim

To format words so that every line has the given maximum width.

Algorithm

1. Pack maximum words into each line.
2. Calculate required spaces.
3. Distribute spaces evenly.
4. Left-justify the last line.

Pseudocode

while words remain:

 take maximum words fitting in line

 if last line or one word:

 left justify

 else:

 distribute spaces evenly

Time Complexity

$O(n)$

Space Complexity

$O(n)$

Code

```
def justify(words, width):
```

```
    result = []
```

```
    i = 0
```

```

while i < len(words):

    j = i

    length = 0

    while j < len(words) and length + len(words[j]) + (j-i) <= width:

        length += len(words[j])

        j += 1

    line = words[i:j]

    spaces = width - sum(map(len, line))

    if j == len(words) or len(line) == 1:

        result.append(" ".join(line).ljust(width))

    else:

        gaps = len(line)-1

        each = spaces // gaps

        extra = spaces % gaps

        result.append((" " * (each+1)).join(line[:extra+1]) +

                        (" " * each).join(line[extra+1:]))

    i = j

return result

```

```
print(justify(  
    ["This","is","an","example","of","text","justification."], 16))
```

Input

Words and `maxWidth = 16`.

Output

This is an
example of text
justification.

Result

The text was justified successfully to the required width.

11. Prefix and Suffix Search

Aim

To find the largest index of a word having the given prefix and suffix.

Algorithm

1. Store all words.
2. Check every word for the prefix and suffix.
3. Update the largest matching index.
4. Return the index.

Pseudocode

```
answer = -1

for each word:

    if word starts with prefix and ends with suffix:

        answer = index

return answer
```

Time Complexity

$O(n \times m)$

Space Complexity

$O(n)$

Code

```
class WordFilter:

    def __init__(self, words):

        self.words = words


    def f(self, pref, suff):

        ans = -1

        for i, word in enumerate(self.words):

            if word.startswith(pref) and word.endswith(suff):

                ans = i

        return ans


w = WordFilter(["apple"])
```

```
print(w.f("a", "e"))
```

Input

```
words = ["apple"], prefix = "a", suffix = "e"
```

Output

0

Result

The correct word index was found successfully.

12. Floyd's Algorithm – All Pairs Shortest Path

Aim

To find the shortest distance between all pairs of cities using Floyd's Algorithm.

Algorithm

1. Create the distance matrix.
2. Select each city as an intermediate city.
3. Update distances using `dist[i][j]`.
4. Display the final matrix.

Pseudocode

```
for k = 0 to n-1:
```

```
    for i = 0 to n-1:
```

```
        for j = 0 to n-1:
```

$$\text{dist}[i][j] = \min(\text{dist}[i][j], \\ \text{dist}[i][k] + \text{dist}[k][j])$$

Time Complexity

$O(n^3)$

Space Complexity

$O(n^2)$

Code

```
INF = float('inf')
```

```
dist = [  
    [0,3,8,-4],  
    [INF,0,4,1],  
    [2,INF,0,INF],  
    [INF,6,-5,0]  
]
```

```
for k in range(4):
```

```
    for i in range(4):
```

```
        for j in range(4):
```

```
            dist[i][j] = min(dist[i][j],  
                             dist[i][k] + dist[k][j])
```

```
for row in dist:
```

```
    print(row)
```

```
print("City 1 to City 3 =", dist[0][2])
```

Input

Given city distance matrix.

Output

City 1 to City 3 = -9

Result

The shortest path between all city pairs was calculated successfully.

13. Floyd's Algorithm with Link Failure

Aim

To find the shortest path between routers before and after a link failure.

Algorithm

1. Create the router distance matrix.
2. Apply Floyd's Algorithm.
3. Remove the B-D link.
4. Apply Floyd's Algorithm again.
5. Compare the shortest paths.

Pseudocode

create distance matrix

apply Floyd algorithm

store A to F distance

remove B-D link

apply Floyd algorithm again

display new A to F distance

Time Complexity

$O(n^3)$

Space Complexity

$O(n^2)$

Code

INF = 999

```
def floyd(d):
```

```
    n = len(d)
```

```
    for k in range(n):
```

```
        for i in range(n):
```

```
            for j in range(n):
```

```
                d[i][j] = min(d[i][j], d[i][k] + d[k][j])
```

```
    return d
```

```
d = [  
    [0,1,5,INF,INF,INF],  
    [1,0,2,1,INF,INF],  
    [5,2,0,INF,3,INF],  
    [INF,1,INF,0,1,6],  
    [INF,INF,3,1,0,2],  
    [INF,INF,INF,6,2,0]  
]  
  
print("Before failure:", floyd([r[:] for r in d])[0][5])
```

```
d[1][3] = d[3][1] = INF
```

```
print("After failure:", floyd([r[:] for r in d])[0][5])
```

Input

Router network and B-D link failure.

Output

Before failure: 5

After failure: 10

Result

The shortest path was updated successfully after the link failure.

14. Floyd's Algorithm – City Network

Aim

To find shortest paths between all cities using Floyd's Algorithm.

Algorithm

1. Create the distance matrix.
2. Apply Floyd's Algorithm.
3. Count cities within the threshold.
4. Select the city with the required minimum count.

Pseudocode

apply Floyd algorithm

for every city:

 count reachable cities

select city with minimum count

if tie, select greatest city number

Time Complexity

$O(n^3)$

Space Complexity

$O(n^2)$

Code

```
def city(n, edges, threshold):
```

```
    INF = 10**9
```

```
d = [[INF]*n for _ in range(n)]
```

```
for i in range(n):
```

```
    d[i][i] = 0
```

```
for a,b,w in edges:
```

```
    d[a][b] = d[b][a] = w
```

```
for k in range(n):
```

```
    for i in range(n):
```

```
        for j in range(n):
```

```
            d[i][j] = min(d[i][j], d[i][k]+d[k][j])
```

```
counts = []
```

```
for i in range(n):
```

```
    counts.append(sum(d[i][j] <= threshold
```

```
        for j in range(n) if i != j))
```

```
return min(range(n), key=lambda x: (counts[x], -x))
```

```
print(city(5, [[0,1,2],[0,4,8],[1,2,3],
```

```
    [1,4,2],[2,3,1],[3,4,1]], 2))
```

Input

City graph and distance threshold.

Output

0

Result

The required city was identified successfully.

15. Optimal Binary Search Tree

Aim

To construct an Optimal Binary Search Tree with minimum search cost.

Algorithm

1. Store keys and frequencies.
2. Calculate frequency sums.
3. Try each key as root.
4. Select the minimum cost.
5. Store the root in the root table.

Pseudocode

for length = 1 to n:

 for each interval:

 cost = infinity

 for each key as root:

 calculate left + right + frequency sum

choose minimum

Time Complexity

$O(n^3)$

Space Complexity

$O(n^2)$

Code

```
def obst(freq):  
    n = len(freq)  
    dp = [[0]*n for _ in range(n)]  
  
    for i in range(n):  
        dp[i][i] = freq[i]  
  
    for length in range(2,n+1):  
        for i in range(n-length+1):  
            j = i+length-1  
            total = sum(freq[i:j+1])  
            dp[i][j] = min(  
                dp[i][r-1] if r > i else 0  
                + dp[r+1][j] if r < j else 0  
                + total  
                for r in range(i,j+1)
```

```
)  
return dp[0][n-1]  
  
print(obst([0.1,0.2,0.4,0.3]))
```

Input

Keys = {A, B, C, D}
Frequencies = {0.1, 0.2, 0.4, 0.3}

Output

1.7

Result

The Optimal Binary Search Tree was constructed with minimum cost **1.7**.

16. OBST with Keys and Frequencies

Aim

To construct an Optimal Binary Search Tree for the given keys and frequencies.

Algorithm

1. Initialize the cost table.
2. Calculate costs for different roots.
3. Choose the minimum cost.
4. Store the root information.

Pseudocode

for every interval:

try every key as root

cost = left cost + right cost + frequency sum

select minimum cost

Time Complexity

$O(n^3)$

Space Complexity

$O(n^2)$

Code

```
def obst(freq):
```

```
    n = len(freq)
```

```
    dp = [[0]*n for _ in range(n)]
```

```
    for i in range(n):
```

```
        dp[i][i] = freq[i]
```

```
    for length in range(2,n+1):
```

```
        for i in range(n-length+1):
```

```
            j = i+length-1
```

```
            total = sum(freq[i:j+1])
```

```
            dp[i][j] = min(
```

```
                (dp[i][r-1] if r>i else 0) +
```



```

        (dp[r+1][j] if r<j else 0) + total
    for r in range(i,j+1)
)
return dp[0][n-1]

print(obst([4,2,6,3]))

```

Input

Keys = {10, 12, 16, 21}
 Frequencies = {4, 2, 6, 3}

Output

26

Result

The minimum OBST cost was found as **26**.

17. Cat and Mouse Game

Aim

To determine whether the Mouse wins, Cat wins, or the game ends in a draw.

Algorithm

1. Represent each state as (mouse, cat, turn).
2. Check winning and losing states.
3. Explore possible moves.

4. Use previously calculated states to avoid repetition.
5. Return the game result.

Pseudocode

if mouse == 0: Mouse wins

if mouse == cat: Cat wins

if state repeats: Draw

check all possible moves

return optimal result

Time Complexity

$O(V^2)$

Space Complexity

$O(V^2)$

Code

```
from functools import lru_cache
```

```
def cat_mouse(graph):
```

```
    @lru_cache(None)
```

```
    def game(mouse, cat, turn):
```

```
        if mouse == 0:
```

```
            return 1
```

```
        if mouse == cat:
```

```
            return 2
```

```

for nxt in graph[mouse if turn == 0 else cat]:
    if turn == 1 and nxt == 0:
        continue

    if turn == 0:
        if game(nxt, cat, 1) == 1:
            return 1
    else:
        if game(mouse, nxt, 0) == 2:
            return 2

return 0

return game(1,2,0)

print(cat_mouse([[2,5],[3],[0,4,5],[1,4,5],
                [2,3],[0,2,3]]))

```

Input

Given undirected graph.

Output

0

Result

The game result was determined successfully.

18. Maximum Probability Path

Aim

To find the path having the maximum probability of success.

Algorithm

1. Create the graph.
2. Store probabilities.
3. Update the maximum probability of reaching each node.
4. Continue until the destination is reached.

Pseudocode

```
prob[start] = 1
```

```
while nodes remain:
```

```
    choose node with maximum probability
```

```
    update neighboring probabilities
```

```
return prob[end]
```

Time Complexity

$O(E \log V)$

Space Complexity

$O(V + E)$

Code

```
import heapq

def max_probability(n, edges, prob, start, end):
    g = [[] for _ in range(n)]

    for (a,b),p in zip(edges,prob):
        g[a].append((b,p))
        g[b].append((a,p))

    best = [0]*n
    best[start] = 1
    pq = [(-1,start)]

    while pq:
        p,u = heapq.heappop(pq)
        p = -p

        if u == end:
            return p

        for v,w in g[u]:
            new = p*w
```

```
        if new > best[v]:
            best[v] = new
            heapq.heappush(pq,(-new,v))

    return 0

print(max_probability(
    3, [[0,1],[1,2],[0,2]],
    [0.5,0.5,0.2], 0, 2))
```

Input

Graph, success probabilities, start and end nodes.

Output

0.25

Result

The maximum probability path was found successfully.

19. Unique Paths in a Grid

Aim

To find the number of unique paths from the top-left to the bottom-right of a grid.

Algorithm

1. Create a DP table.
2. Set the first row and column to 1.
3. Add top and left values for each cell.
4. Return the last cell value.

Pseudocode

`dp[0][0] = 1`

for every cell:

`dp[i][j] = dp[i-1][j] + dp[i][j-1]`

`return dp[m-1][n-1]`

Time Complexity

$O(m \times n)$

Space Complexity

$O(m \times n)$

Code

```
def paths(m,n):
```

```
    dp = [[1]*n for _ in range(m)]
```

```
    for i in range(1,m):
```

```
        for j in range(1,n):
```

```
            dp[i][j] = dp[i-1][j] + dp[i][j-1]
```

```
    return dp[m-1][n-1]
```

```
print(paths(3,7))
```

```
print(paths(3,2))
```

Input

```
m = 3, n = 7
```

```
m = 3, n = 2
```

Output

```
28
```

```
3
```

Result

The number of unique paths was calculated successfully.

20. Number of Good Pairs

Aim

To count pairs of equal elements where $i < j$.

Algorithm

1. Store the frequency of each number.
2. For every repeated number, add its previous frequency to the answer.
3. Update its frequency.

Pseudocode

```
ans = 0
```

```
for number in array:
```



```
    ans += frequency[number]

    frequency[number]++

return ans
```

Time Complexity

$O(n)$

Space Complexity

$O(n)$

Code

```
def good_pairs(nums):

    freq = {}

    ans = 0

    for x in nums:

        ans += freq.get(x,0)

        freq[x] = freq.get(x,0) + 1

    return ans

print(good_pairs([1,2,3,1,1,3]))

print(good_pairs([1,1,1,1]))
```

Input

```
[1,2,3,1,1,3]
[1,1,1,1]
```

Output

4

6

Result

The number of good pairs was found successfully.

21. City With Smallest Number of Neighbors

Aim

To find the city having the smallest number of reachable cities within a threshold.

Algorithm

1. Create the distance matrix.
2. Apply Floyd's Algorithm.
3. Count reachable cities for each city.
4. Select the city with minimum count.
5. In case of a tie, select the greatest index.

Pseudocode

apply Floyd algorithm

for every city:

 count cities with distance $\leq$ threshold

return city with minimum count

if tie, return greatest city

Time Complexity

$O(n^3)$

Space Complexity

$O(n^2)$

Code

```
def find_city(n, edges, threshold):
```

```
    INF = 9999
```

```
    d = [[INF]*n for _ in range(n)]
```

```
    for i in range(n):
```

```
        d[i][i] = 0
```

```
    for a,b,w in edges:
```

```
        d[a][b] = d[b][a] = w
```

```
    for k in range(n):
```

```
        for i in range(n):
```

```
            for j in range(n):
```

```
                d[i][j] = min(d[i][j], d[i][k]+d[k][j])
```

```

ans = -1

best = INF

for i in range(n):
    count = sum(d[i][j] <= threshold
                for j in range(n) if i != j)

    if count <= best:
        best = count
        ans = i

return ans

print(find_city(
    4,
    [[0,1,3],[1,2,1],[1,3,4],[2,3,1]],
    4))

```

Input

City graph and distance threshold.

Output

3

Result

The required city was found successfully.

22. Network Delay Time

Aim

To find the minimum time required for all nodes to receive a signal.

Algorithm

1. Create a directed weighted graph.
2. Apply Dijkstra's Algorithm from source k .
3. Find the maximum shortest distance.
4. If any node is unreachable, return -1 .

Pseudocode

distance[k] = 0

insert k into priority queue

while queue not empty:

 select minimum distance node

 update its neighbours

if any node is unreachable:

 return -1

return maximum distance

Time Complexity

$O((V + E) \log V)$

Space Complexity

$O(V + E)$

Code

```
import heapq

def network_delay(times, n, k):

    g = [[] for _ in range(n+1)]

    for u,v,w in times:

        g[u].append((v,w))

    dist = [float('inf')] * (n+1)

    dist[k] = 0

    pq = [(0,k)]

    while pq:

        d,u = heapq.heappop(pq)

        if d > dist[u]:

            continue
```

```

for v,w in g[u]:
    nd = d+w
    if nd < dist[v]:
        dist[v] = nd
        heapq.heappush(pq,(nd,v))

ans = max(dist[1:])

return -1 if ans == float('inf') else ans

print(network_delay([[2,1,1],[2,3,1],[3,4,1]],4,2))
print(network_delay([[1,2,1]],2,1))
print(network_delay([[1,2,1]],2,2))

```

Input

Directed edges, number of nodes and starting node.

Output

2

1

-1

Result

The minimum network delay time was calculated successfully.